Лабораторна робота №2

Тема: Породжувальні шаблони

Мета роботи: навчитися реалізовувати породжувальні шаблони проєктування

**Завдання 1: Фабричний метод.**

1. Напишіть систему класів для реалізації функціоналу створення різних типів підписок для відео провайдера.
2. Кожна з підписок повинна мати щомісячну плату, мінімальний період підписки та список каналів й інших можливостей.
3. Види підписок: ***DomesticSubscription, EducationalSubscription, PremiumSubscription***.
4. Придбати (тобто створити) підписку можна за допомогою трьох різних класів: ***WebSite, MobileApp, ManagerCall***, кожен з них має реалізувати свою логіку створення підписок.
5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.
6. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою https://app.diagrams.net/, експортуйте та завантажте її до репозиторія.

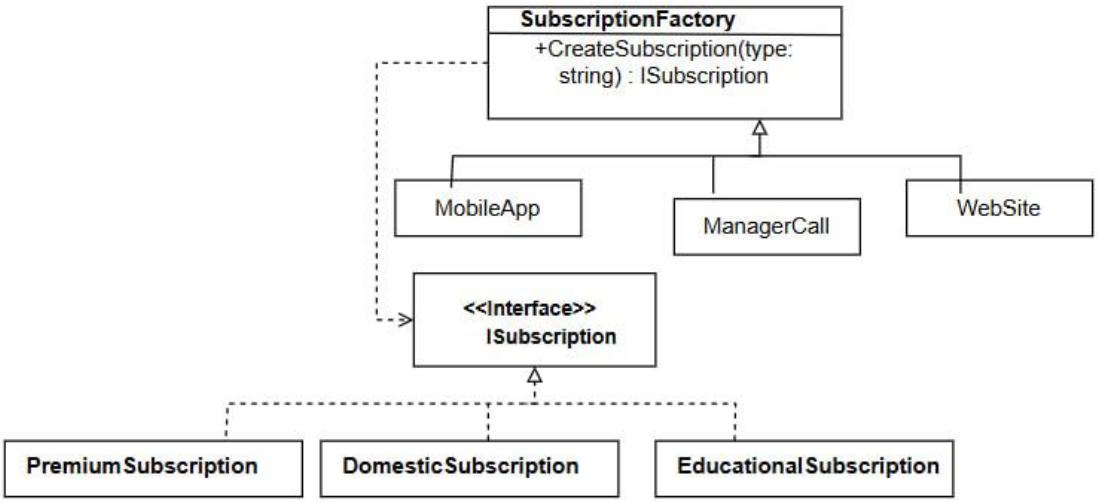


Рис. 1

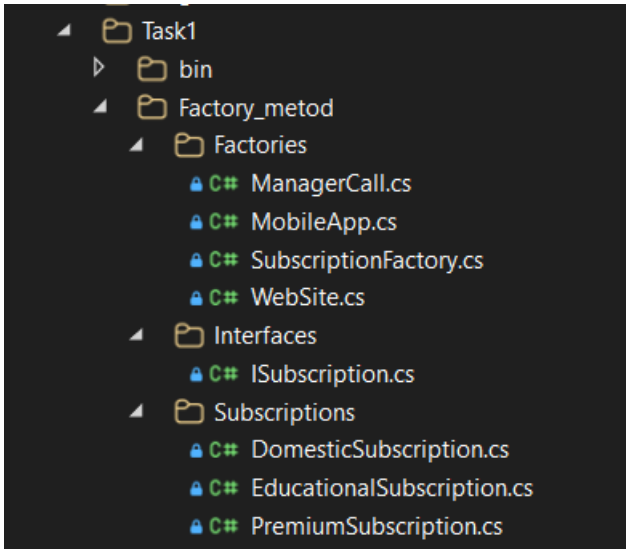


Рис. 2

Лістинг програми:

```
using System;
using lab1.Factory_metod.Subscriptions;
using lab1.Factory_metod.Interfaces;
using lab1.Factory_metod.Factories;

namespace lab1
{
    class Program
    {

        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8; // Для коректних літер 'і',
'ї', 'є'

            // Масив різних фабрик для перевірки всіх каналів продажу
            SubscriptionFactory[] factories = new SubscriptionFactory[]
            {
        new WebSite(),
        new MobileApp(),
        new ManagerCall()
            };

            string[] types = { "domestic", "educational", "premium" };

            Console.WriteLine("======= ТЕСТУВАННЯ СИСТЕМИ ПІДПИСОК =======");

            foreach (var factory in factories)
            {
                Console.WriteLine($"\n>>> Створення через: {factory.GetType().Name}");

                foreach (var type in types)
                {
                    try
                    {
                        ISubscription sub = factory.CreateSubscription(type);
                        Console.WriteLine($"[{type}] - Ціна: {sub.MonthlyFee} грн, Каналів:
{sub.Channels.Count}");
                    }
                    catch (Exception ex)
                    {
                        // Тут ми побачимо помилку для MobileApp, де ми спеціально не додали
educational
                        Console.WriteLine($"[{type}] - Помилка: {ex.Message}");
                    }
                }
            }

            Console.WriteLine("\n==========");
```

```
            Console.WriteLine("Натисніть будь-яку клавішу для виходу...");
            Console.ReadKey();
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using lab1.Factory_metod.Interfaces;
using lab1.Factory_metod.Subscriptions;

namespace lab1.Factory_metod.Factories
{
    public class ManagerCall : SubscriptionFactory
    {
        public override ISubscription CreateSubscription(string type)
        {
            // Використовуємо класичний switch для сумісності з C# 7.3
            switch (type.ToLower())
            {
                case "domestic":
                    return new DomesticSubscription();
                case "educational":
                    return new EducationalSubscription();
                case "premium":
                    return new PremiumSubscription();
                default:
                    throw new ArgumentException("Менеджер не знайшов таку підписку");
            }
        }
    }
}
using lab1.Factory_metod.Interfaces;
using lab1.Factory_metod.Subscriptions;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace lab1.Factory_metod.Factories
{
    public class MobileApp : SubscriptionFactory
    {
        public override ISubscription CreateSubscription(string type)
        {

            switch (type.ToLower())
            {
                case "domestic":
                    return new DomesticSubscription();
                case "premium":
                    return new PremiumSubscription();
                default:
                    throw new ArgumentException("Цей тип недоступний у додатку");
            }
        }
    }
}
using lab1.Factory_metod.Interfaces;

namespace lab1.Factory_metod.Factories
{
    public abstract class SubscriptionFactory
    {
        // Фабричний метод
        public abstract ISubscription CreateSubscription(string type);
    }
}
using lab1.Factory_metod.Interfaces;
using lab1.Factory_metod.Subscriptions;
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace lab1.Factory_metod.Factories
{
    public class WebSite : SubscriptionFactory
    {
        public override ISubscription CreateSubscription(string type)
        {
            switch (type.ToLower())
            {
                case "domestic":
                    return new DomesticSubscription();
                case "educational":
                    return new EducationalSubscription();
                case "premium":
                    return new PremiumSubscription();
                default:
                    throw new ArgumentException("Невідомий тип");
            }
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace lab1.Factory_metod.Interfaces
{
    public interface ISubscription
    {
        double MonthlyFee { get; }
        int MinimumPeriod { get; }
        List<string> Channels { get; }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using lab1.Factory_metod.Interfaces;

namespace lab1.Factory_metod.Subscriptions
{
    public class DomesticSubscription : ISubscription
    {
        public double MonthlyFee => 120.50; // Твоя ціна
        public int MinimumPeriod => 1;      // Твій термін
        public List<string> Channels => new List<string> { "Discovery", "National Geographic"
};
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using lab1.Factory_metod.Interfaces;

namespace lab1.Factory_metod.Subscriptions
{
    public class EducationalSubscription : ISubscription
    {
        public double MonthlyFee => 50;
        public int MinimumPeriod => 3;      // Твій термін
        public List<string> Channels => new List<string> { "Discovery", "History", "National
Geographic" };
```

```
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using lab1.Factory_metod.Interfaces;

namespace lab1.Factory_metod.Subscriptions
{
  public class PremiumSubscription : ISubscription
    {
        public double MonthlyFee => 200;
        public int MinimumPeriod => 12;
        public List<string> Channels => new List<string> { "Всі канали", "4K Video", "Без
реклами" };
    }
}
```

```
D:\univer\2_course\Konstruyuvannya_PZ\lab2\Task1\bin\Debug\lab1.exe
======= ТЕСТУВАННЯ СИСТЕМИ ПІДПИСОК =======

>>> Створення через: WebSite
[domestic] - Ціна: 120,5 грн, Каналів: 2
[educational] - Ціна: 50 грн, Каналів: 3
[premium] - Ціна: 200 грн, Каналів: 3

>>> Створення через: MobileApp
[domestic] - Ціна: 120,5 грн, Каналів: 2
[educational] - Помилка: Цей тип недоступний у додатку
[premium] - Ціна: 200 грн, Каналів: 3

>>> Створення через: ManagerCall
[domestic] - Ціна: 120,5 грн, Каналів: 2
[educational] - Ціна: 50 грн, Каналів: 3
[premium] - Ціна: 200 грн, Каналів: 3

==========================================
Натисніть будь-яку клавішу для виходу...
```

Рис. 3

**Завдання 2: Абстрактна фабрика.**

1. Створіть фабрику виробництва техніки.
2. На фабриці мають створюватися різні девайси (наприклад, Laptop, Netbook, EBook, Smartphone) для різних брендів (IProne, Kiaomi, Balaxy).
3. Покажіть правильність роботи свого коду запустивши його в головному методі програми.
4. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою https://app.diagrams.net/, експортуйте та завантажте її до репозиторія.

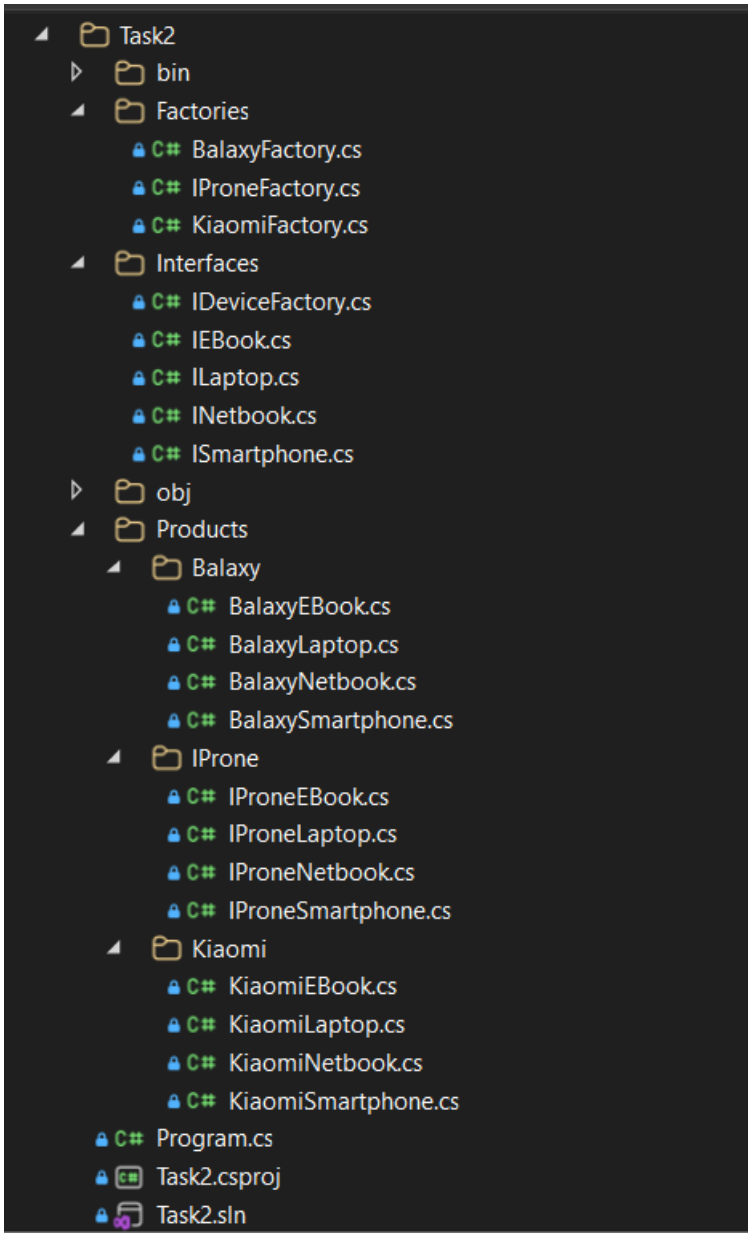


Рис. 4

Лістинг програми:

```
using Task2.Interfaces;
using Task2.Products.Balaxy;

namespace Task2.Factories
{
    public class BalaxyFactory : IDeviceFactory
    {
        public ILaptop CreateLaptop() => new BalaxyLaptop();
        public INetbook CreateNetbook() => new BalaxyNetbook();
        public IEBook CreateEBook() => new BalaxyEBook();
        public ISmartphone CreateSmartphone() => new BalaxySmartphone();
    }
}
using Task2.Interfaces;
using Task2.Products.IProne;

namespace Task2.Factories
{
    public class IProneFactory : IDeviceFactory
    {
        public ILaptop CreateLaptop() => new IProneLaptop();
        public INetbook CreateNetbook() => new IProneNetbook();
        public IEBook CreateEBook() => new IProneEBook();
        public ISmartphone CreateSmartphone() => new IProneSmartphone();
```

```
    }
}
using Task2.Interfaces;
using Task2.Products.Kiaomi;

namespace Task2.Factories
{
    public class KiaomiFactory : IDeviceFactory
    {
        public ILaptop CreateLaptop() => new KiaomiLaptop();
        public INetbook CreateNetbook() => new KiaomiNetbook();
        public IEBook CreateEBook() => new KiaomiEBook();
        public ISmartphone CreateSmartphone() => new KiaomiSmartphone();
    }
}
namespace Task2.Interfaces
{
    public interface IDeviceFactory
    {
        ILaptop CreateLaptop();
        INetbook CreateNetbook();
        IEBook CreateEBook();
        ISmartphone CreateSmartphone();
    }
}
namespace Task2.Interfaces
{
    public interface IEBook
    {
        void OpenBook();
    }
}
namespace Task2.Interfaces
{
    public interface ILaptop
    {
        void GetSpecs();
        string GetModelName();
    }
}
namespace Task2.Interfaces
{
    public interface INetbook
    {
        void OpenWebBrowser();
    }
}
namespace Task2.Interfaces
{
    public interface ISmartphone
    {
        void MakeCall();
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.Balaxy
{
    public class BalaxyEBook : IEBook
    {
        public void OpenBook() => Console.WriteLine("[Balaxy EBook] Читання книги з
комфортного E-Ink екрану...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.Balaxy
{
    public class BalaxyLaptop : ILaptop
    {
        public string GetModelName() => "Balaxy Book Ultra";
```

```
        public void GetSpecs() => Console.WriteLine($"[{GetModelName()}] 32GB RAM, Intel i9");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.Balaxy
{
    public class BalaxyNetbook : INetbook
    {
        public void OpenWebBrowser() => Console.WriteLine("[Balaxy Netbook] Відкриття браузера
Samsung Internet...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.Balaxy
{
    public class BalaxySmartphone : ISmartphone
    {
        public void MakeCall() => Console.WriteLine("[Balaxy Smartphone] Здійснення
надшвидкого дзвінка через мережу 5G...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.IProne
{
    public class IProneEBook : IEBook
    {
        public void OpenBook() => Console.WriteLine("[IProne EBook] Читання книги у додатку
iBooks...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.IProne
{
    public class IProneLaptop : ILaptop
    {
        public string GetModelName() => "IProne Book Pro";
        public void GetSpecs() => Console.WriteLine($"[{GetModelName()}] 16GB RAM, M2 Chip");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.IProne
{
    public class IProneNetbook : INetbook
    {
        public void OpenWebBrowser() => Console.WriteLine("[IProne Netbook] Відкриття браузера
Safari...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.IProne
{
    public class IProneSmartphone : ISmartphone
    {
        public void MakeCall() => Console.WriteLine("[IProne Smartphone] Здійснення дзвінка
через FaceTime...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.IProne
```

```
{
    public class IProneSmartphone : ISmartphone
    {
        public void MakeCall() => Console.WriteLine("[IProne Smartphone] Здійснення дзвінка
через FaceTime...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.IProne
{
    public class IProneSmartphone : ISmartphone
    {
        public void MakeCall() => Console.WriteLine("[IProne Smartphone] Здійснення дзвінка
через FaceTime...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.Kiaomi
{
    public class KiaomiNetbook : INetbook
    {
        public void OpenWebBrowser() => Console.WriteLine("[Kiaomi Netbook] Відкриття браузера
Google Chrome...");
    }
}
using System;
using Task2.Interfaces;

namespace Task2.Products.Kiaomi
{
    public class KiaomiSmartphone : ISmartphone
    {
        public void MakeCall() => Console.WriteLine("[Kiaomi Smartphone] Здійснення звичайного
мобільного дзвінка...");
    }
}
using System;
using Task2.Interfaces;
using Task2.Factories;

namespace Task2
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            Console.WriteLine("=== Виробництво IProne ===");
            TestDeviceProduction(new IProneFactory());

            Console.WriteLine("\n=== Виробництво Kiaomi ===");
            TestDeviceProduction(new KiaomiFactory());

            Console.WriteLine("\n=== Виробництво Balaxy ===");
            TestDeviceProduction(new BalaxyFactory());

            Console.ReadLine();
        }

        // Цей метод працює лише з інтерфейсами. Він не знає, які саме
        // конкретні класи сюди передаються. Це і є магія Абстрактної фабрики.
        static void TestDeviceProduction(IDeviceFactory factory)
        {
            // 1. Створюємо продукти
            ILaptop laptop = factory.CreateLaptop();
            INetbook netbook = factory.CreateNetbook();
            IEBook ebook = factory.CreateEBook();
            ISmartphone smartphone = factory.CreateSmartphone();
```

```
            // 2. Демонструємо їхню роботу
            laptop.GetSpecs();
            netbook.OpenWebBrowser();
            ebook.OpenBook();
            smartphone.MakeCall();
        }
    }
}
```

```
D:\univer\2_course\Konstruyuvannya_PZ\lab2\Task2\bin\Debug\net472\Task2.exe
=== Виробництво IProne ===
[IProne Book Pro] 16GB RAM, M2 Chip
[IProne Netbook] Відкриття браузера Safari...
[IProne EBook] Читання книги у додатку iBooks...
[IProne Smartphone] Здійснення дзвінка через FaceTime...

=== Виробництво Kiaomi ===
[Kiaomi MiBook] 8GB RAM, Intel i5
[Kiaomi Netbook] Відкриття браузера Google Chrome...
[Kiaomi EBook] Читання книги у додатку MiReader...
[Kiaomi Smartphone] Здійснення звичайного мобільного дзвінка...

=== Виробництво Balaxy ===
[Balaxy Book Ultra] 32GB RAM, Intel i9
[Balaxy Netbook] Відкриття браузера Samsung Internet...
[Balaxy EBook] Читання книги з комфортного E-Ink екрану...
[Balaxy Smartphone] Здійснення надшвидкого дзвінка через мережу 5G...
```

Рис. 5

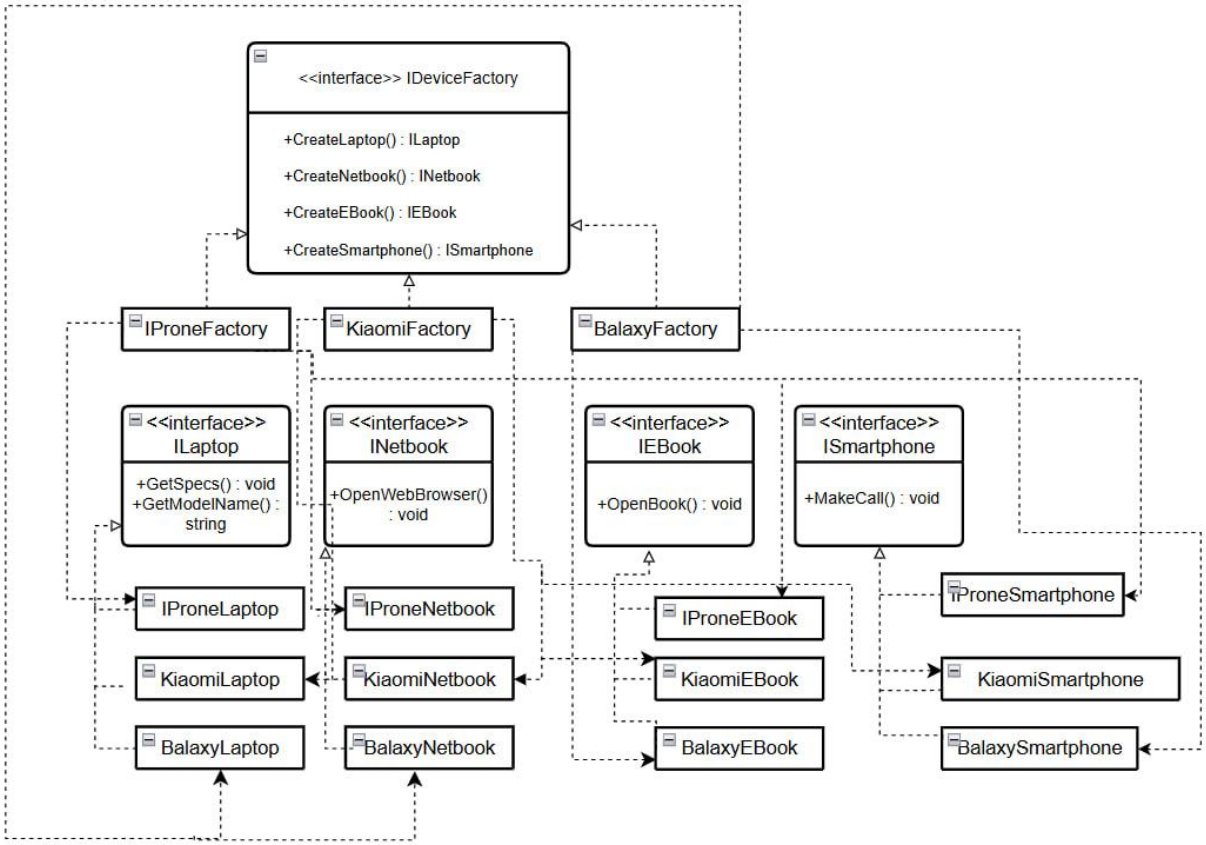


Рис. 6

## Завдання 3: Одинак.

1. Створіть клас ***Authenticator*** таким чином, щоб бути впевненим, що цей клас може створити лише один екземпляр, незалежно від кількості потоків і класів, що його наслідують.
2. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
using System;

namespace Task3
{
        // Ключове слово 'sealed' забороняє наслідування цього класу.
        public sealed class Authenticator
        {
                // Приховане поле, в якому зберігатиметься єдиний екземпляр
                private static Authenticator _instance;

                // Спеціальний об'єкт-заглушка для синхронізації потоків
                private static readonly object _lockObject = new object();

                // Приватний конструктор
                private Authenticator()
                {
                        Console.WriteLine(" ---> [СИСТЕМА] Викликано конструктор
Authenticator. Створено новий екземпляр!");
                }

                // Публічна точка доступу до нашого екземпляра
                public static Authenticator Instance
                {
                        get
                        {
                                // Подвійна перевірка блокування (Double-Check Locking)
                                if (_instance == null)
                                {
                                        lock (_lockObject)
                                        {
                                                if (_instance == null)
                                                {
                                                        _instance = new Authenticator();
                                                }
                                        }
                                }
                                return _instance;
                        }
                }

                // Звичайний метод класу для демонстрації роботи
                public void AuthenticateUser(string username)
                {
                        Console.WriteLine($"[Authenticator] Користувач '{username}' успішно
пройшов перевірку.");
                }
        }
}
```

```
using System;
using System.Threading.Tasks;

namespace Task3
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Console.WriteLine("=== Старт тестування патерну Одинак (Singleton) ===\n");

            // Створюємо масив завдань (потоків), щоб імітувати одночасний доступ
            Task[] threads = new Task[5];

            for (int i = 0; i < 5; i++)
            {
                int threadId = i + 1;

                // Запускаємо паралельний потік
                threads[i] = Task.Run(() =>
                {
                    // Отримуємо доступ до нашого Одинака
                    Authenticator auth = Authenticator.Instance;

                    auth.AuthenticateUser($"User_{threadId}");
                    Console.WriteLine($"Потік {threadId} отримав екземпляр з HashCode:
{auth.GetHashCode()}");
                });
            }

            // Чекаємо, поки всі паралельні потоки завершать свою роботу
            Task.WaitAll(threads);

            Console.WriteLine("\n=== Перевірка у головному потоці ===");
            Authenticator auth1 = Authenticator.Instance;
            Authenticator auth2 = Authenticator.Instance;

            // Перевіряємо, чи це дійсно один і той самий об'єкт у пам'яті
            if (ReferenceEquals(auth1, auth2))
            {
                Console.WriteLine("РЕЗУЛЬТАТ: Змінні auth1 та auth2 повністю ідентичні.");
                Console.WriteLine("Патерн працює бездоганно: існує лише один екземпляр класу
Authenticator!");
            }

            Console.ReadLine();
        }
    }
}
```

```
D:\univer\2_course\Konstruyuvannya_PZ\lab2\Task3\bin\Debug\net472\Task3.exe
=== Старт тестування патерну Одинак (Singleton) ===

 ---> [СИСТЕМА] Викликано конструктор Authenticator. Створено новий екземпляр!
[Authenticator] Користувач 'User_1' успішно пройшов перевірку.
Потік 1 отримав екземпляр з HashCode: 6044116
[Authenticator] Користувач 'User_3' успішно пройшов перевірку.
Потік 3 отримав екземпляр з HashCode: 6044116
[Authenticator] Користувач 'User_4' успішно пройшов перевірку.
Потік 4 отримав екземпляр з HashCode: 6044116
[Authenticator] Користувач 'User_5' успішно пройшов перевірку.
[Authenticator] Користувач 'User_2' успішно пройшов перевірку.
Потік 2 отримав екземпляр з HashCode: 6044116
Потік 5 отримав екземпляр з HashCode: 6044116

=== Перевірка у головному потоці ===
РЕЗУЛЬТАТ: Змінні auth1 та auth2 повністю ідентичні.
Патерн працює бездоганно: існує лише один екземпляр класу Authenticator!
```

Рис. 7

**Завдання 4: Прототип.**

1. Створіть клас ***Virus***. Він повинен містити вагу, вік, ім'я, вид і масив дітей, екземплярів ***Virus***.
2. Створіть екземпляри для цілого "сімейства" вірусів (мінімум три покоління).
3. За допомогою шаблону Прототип реалізуйте можливість клонування наявних вірусів.
4. При клонуванні віруса-батька повинні клонуватися всі його діти.
5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
using System;
using System.Collections.Generic;

namespace Task4
{
    public interface IPrototype
    {
        // Метод Clone повертає новий екземпляр такого ж типу
        Virus Clone();
    }

    public class Virus : IPrototype
    {
        public double Weight { get; set; }
        public int Age { get; set; }
        public string Name { get; set; }
        public string Type { get; set; }

        // Масив дітей (використовуємо List для зручності роботи з колекцією)
        public List<Virus> Children { get; set; }

        // Конструктор
        public Virus(double weight, int age, string name, string type)
        {
            Weight = weight;
            Age = age;
            Name = name;
            Type = type;
            Children = new List<Virus>(); // Ініціалізуємо порожній список при створенні
        }

        // Реалізація методу клонування
        public Virus Clone()
        {
            // Використовуємо вбудований механізм (MemberwiseClone)
            Virus clone = (Virus)this.MemberwiseClone();

            // 2. Робимо ГЛИБОКЕ копіювання для дітей (виконання пункту №4 завдання)
            // Якщо цього не зробити, клон і оригінал будуть ділити один і той самий масив
дітей у пам'яті.
            clone.Children = new List<Virus>();

            foreach (var child in this.Children)
            {
                // Рекурсивно клонуємо кожну дитину і додаємо в новий масив клону
                clone.Children.Add(child.Clone());
            }
```

```
            return clone;
        }

        public void PrintFamilyTree(string indent = "")
        {
            Console.WriteLine($"{indent} Вірус '{Name}' [Вид: {Type} | Вік: {Age} | Вага:
{Weight}]");
            foreach (var child in Children)
            {
                // Додаємо пробіли для візуального відображення поколінь
                child.PrintFamilyTree(indent + "   ");
            }
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            // 1-ше покоління
            Virus grandParentVirus = new Virus(1.5, 10, "Alpha-01", "Коронавірус");

            // 2-ге покоління
            Virus parentVirus1 = new Virus(1.2, 5, "Beta-01", "Коронавірус-Мутант");
            Virus parentVirus2 = new Virus(1.3, 4, "Beta-02", "Коронавірус-Мутант");

            // 3-тє покоління
            Virus childVirus1 = new Virus(0.5, 1, "Gamma-01", "Коронавірус-Штам");
            Virus childVirus2 = new Virus(0.6, 1, "Gamma-02", "Коронавірус-Штам");
            Virus childVirus3 = new Virus(0.4, 0, "Gamma-03", "Коронавірус-Штам");

            // Будуємо родинні зв'язки
            parentVirus1.Children.Add(childVirus1);
            parentVirus1.Children.Add(childVirus2);
            parentVirus2.Children.Add(childVirus3);

            grandParentVirus.Children.Add(parentVirus1);
            grandParentVirus.Children.Add(parentVirus2);

            // --- ДЕМОНСТРАЦІЯ РОБОТИ ---
            Console.WriteLine("=== ОРИГІНАЛЬНА СІМ'Я ВІРУСІВ ===");
            grandParentVirus.PrintFamilyTree();

            Console.WriteLine("\n... Клонування головного вірусу ...\n");

            // Клонуємо дідуся (клонується все дерево)
            Virus clonedFamily = grandParentVirus.Clone();

            Console.WriteLine("=== СКЛОНОВАНА СІМ'Я ВІРУСІВ ===");
            clonedFamily.PrintFamilyTree();

            // --- ПЕРЕВІРКА ГЛИБОКОГО КОПІЮВАННЯ ---
            Console.WriteLine("\n=== ПЕРЕВІРКА (Змінюємо оригінал) ===");
            Console.WriteLine("Змінюємо ім'я оригіналу та додаємо йому нову дитину...");

            grandParentVirus.Name = "ОРИГІНАЛ-МУТАНТ";
            grandParentVirus.Children[0].Children[0].Name = "ЗМІНЕНА ДИТИНА"; // Змінюємо ім'я
дитини в 3-му поколінні
            grandParentVirus.Children.Add(new Virus(0.1, 0, "Новий Несподіваний Вірус",
"Невідомий"));

            Console.WriteLine("\nСтан ОРИГІНАЛУ після змін:");
            grandParentVirus.PrintFamilyTree();

            Console.WriteLine("\nСтан КЛОНУ після змін в оригіналі (має залишитись без
змін!):");
            clonedFamily.PrintFamilyTree();

            Console.ReadLine();
        }
    }
```

}

```
D:\univer\2_course\Konstruyuvannya_PZ\lab2\Task4\bin\Debug\net472\Task4.exe
    Вірус 'Beta-02' [Вид: Коронавірус-Мутант | Вік: 4 | Вага: 1,3]
       Вірус 'Gamma-03' [Вид: Коронавірус-Штам | Вік: 0 | Вага: 0,4]

... Клонування головного вірусу ...

=== СКЛОНОВАНА СІМ'Я ВІРУСІВ ===
 Вірус 'Alpha-01' [Вид: Коронавірус | Вік: 10 | Вага: 1,5]
    Вірус 'Beta-01' [Вид: Коронавірус-Мутант | Вік: 5 | Вага: 1,2]
       Вірус 'Gamma-01' [Вид: Коронавірус-Штам | Вік: 1 | Вага: 0,5]
       Вірус 'Gamma-02' [Вид: Коронавірус-Штам | Вік: 1 | Вага: 0,6]
    Вірус 'Beta-02' [Вид: Коронавірус-Мутант | Вік: 4 | Вага: 1,3]
       Вірус 'Gamma-03' [Вид: Коронавірус-Штам | Вік: 0 | Вага: 0,4]

=== ПЕРЕВІРКА (Змінюємо оригінал) ===
Змінюємо ім'я оригіналу та додаємо йому нову дитину...

Стан ОРИГІНАЛУ після змін:
 Вірус 'ОРИГІНАЛ-МУТАНТ' [Вид: Коронавірус | Вік: 10 | Вага: 1,5]
    Вірус 'Beta-01' [Вид: Коронавірус-Мутант | Вік: 5 | Вага: 1,2]
       Вірус 'ЗМІНЕНА ДИТИНА' [Вид: Коронавірус-Штам | Вік: 1 | Вага: 0,5]
       Вірус 'Gamma-02' [Вид: Коронавірус-Штам | Вік: 1 | Вага: 0,6]
    Вірус 'Beta-02' [Вид: Коронавірус-Мутант | Вік: 4 | Вага: 1,3]
       Вірус 'Gamma-03' [Вид: Коронавірус-Штам | Вік: 0 | Вага: 0,4]
    Вірус 'Новий Несподіваний Вірус' [Вид: Невідомий | Вік: 0 | Вага: 0,1]

Стан КЛОНУ після змін в оригіналі (має залишитись без змін!):
 Вірус 'Alpha-01' [Вид: Коронавірус | Вік: 10 | Вага: 1,5]
    Вірус 'Beta-01' [Вид: Коронавірус-Мутант | Вік: 5 | Вага: 1,2]
       Вірус 'Gamma-01' [Вид: Коронавірус-Штам | Вік: 1 | Вага: 0,5]
       Вірус 'Gamma-02' [Вид: Коронавірус-Штам | Вік: 1 | Вага: 0,6]
```

Рис. 8

**Завдання 5: Будівельник.**

1. Створіть клас ***HeroBuilder***, який буде створювати персонажа гри, поступово додаючи до нього різні ознаки, наприклад зріст, статуру, колір волосся, очей, одяг, інвентар тощо (можете включити фантазію).
2. Створіть клас ***EnemyBuilder***, який буде реалізовувати єдиний інтерфейс з ***HeroBuilder***. Відмінністю між ними можуть бути спеціальні методи для творення добра або зла, а також списки добрих і злих справ відповідно.
3. За допомогою свого білдера і класу-директора створіть героя (або героїню) своєї мрії 🙂, а також свого найзапеклішого ворога.
4. Зверніть увагу, що Ваші білдери повинні реалізовувати текучий интерфейс (fluent interface).
5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.
6. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою https://app.diagrams.net/, експортуйте та завантажте її до репозиторія.

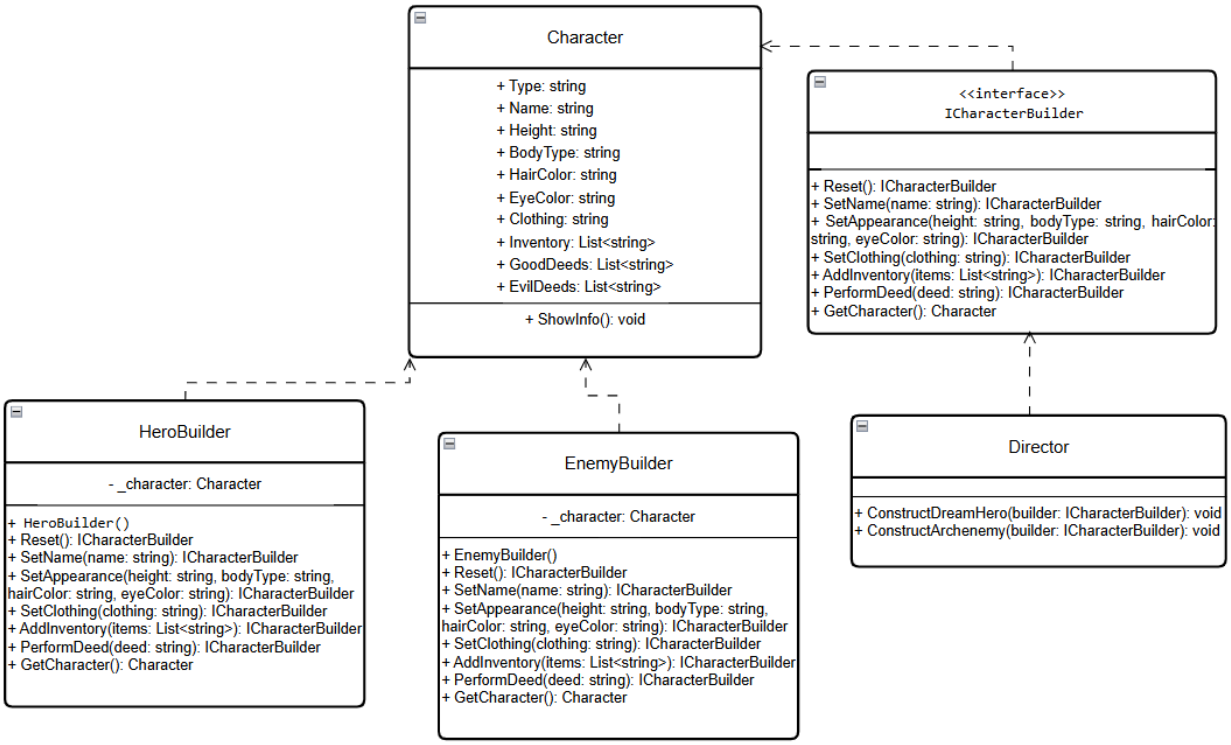


Рис. 9

Лістинг програми:

```
using System.Collections.Generic;
using Task5.Interfaces;
using Task5.Products;

namespace Task5.Builders
{
    public class EnemyBuilder : ICharacterBuilder
    {
        private Character _character;

        public EnemyBuilder() { Reset(); }

        public ICharacterBuilder Reset()
        {
            _character = new Character { Type = "Найзапекліший Ворог" };
            return this;
        }

        public ICharacterBuilder SetName(string name) { _character.Name = name; return this; }
        public ICharacterBuilder SetAppearance(string height, string bodyType, string
hairColor, string eyeColor)
        {
            _character.Height = height; _character.BodyType = bodyType;
            _character.HairColor = hairColor; _character.EyeColor = eyeColor;
            return this;
        }
        public ICharacterBuilder SetClothing(string clothing) { _character.Clothing =
clothing; return this; }
        public ICharacterBuilder AddInventory(List<string> items) {
_character.Inventory.AddRange(items); return this; }

        // Специфічна логіка: Ворог робить лише злі справи
        public ICharacterBuilder PerformDeed(string deed)
        {
            _character.EvilDeeds.Add(deed);
            return this;
        }

        public Character GetCharacter() => _character;
```

```
    }
}
using System.Collections.Generic;
using Task5.Interfaces;
using Task5.Products;

namespace Task5.Builders
{
    public class HeroBuilder : ICharacterBuilder
    {
        private Character _character;

        public HeroBuilder() { Reset(); }

        public ICharacterBuilder Reset()
        {
            _character = new Character { Type = "Герой" };
            return this;
        }

        public ICharacterBuilder SetName(string name) { _character.Name = name; return this; }
        public ICharacterBuilder SetAppearance(string height, string bodyType, string
hairColor, string eyeColor)
        {
            _character.Height = height; _character.BodyType = bodyType;
            _character.HairColor = hairColor; _character.EyeColor = eyeColor;
            return this;
        }
        public ICharacterBuilder SetClothing(string clothing) { _character.Clothing =
clothing; return this; }
        public ICharacterBuilder AddInventory(List<string> items) {
_character.Inventory.AddRange(items); return this; }

        // Специфічна логіка: Герой робить лише добрі справи
        public ICharacterBuilder PerformDeed(string deed)
        {
            _character.GoodDeeds.Add(deed);
            return this;
        }

        public Character GetCharacter() => _character;
    }
}
using System.Collections.Generic;
using Task5.Interfaces;

namespace Task5.Directors
{
    public class Director
    {
        // Метод для створення Героїні мрії
        public void ConstructDreamHero(ICharacterBuilder builder)
        {
            builder.Reset()
                   .SetName("Чародійка")
                   .SetAppearance("Середній зріст", "Спортивна", "Каштанове", "Голубі")
                   .SetClothing("Сарафан")
                   .AddInventory(new List<string> { "Ноутбук", "Зілля" })
                   .PerformDeed("Врятувала світ")
                   .PerformDeed("Зробила всі лаби");
        }

        // Метод для створення Ворога
        public void ConstructArchenemy(ICharacterBuilder builder)
        {
            builder.Reset()
                   .SetName("Лінька")
                   .SetAppearance("Височезний", "Кремезний", "Чорне", "Червоні")
                   .SetClothing("Темна піксельна броня")
                   .AddInventory(new List<string> { "Блоки TNT", "Зілля шкоди" })
                   .PerformDeed("Зруйнував чужу базу")
                   .PerformDeed("Стер збереження гри");
        }
```

```
    }
}
using System.Collections.Generic;
using Task5.Products;

namespace Task5.Interfaces
{
    public interface ICharacterBuilder
    {
        ICharacterBuilder Reset();
        ICharacterBuilder SetName(string name);
        ICharacterBuilder SetAppearance(string height, string bodyType, string hairColor,
string eyeColor);
        ICharacterBuilder SetClothing(string clothing);
        ICharacterBuilder AddInventory(List<string> items);
        ICharacterBuilder PerformDeed(string deed); // Загальний метод для справ
        Character GetCharacter();
    }
}
using System.Collections.Generic;
using Task5.Products;

namespace Task5.Interfaces
{
    public interface ICharacterBuilder
    {
        ICharacterBuilder Reset();
        ICharacterBuilder SetName(string name);
        ICharacterBuilder SetAppearance(string height, string bodyType, string hairColor,
string eyeColor);
        ICharacterBuilder SetClothing(string clothing);
        ICharacterBuilder AddInventory(List<string> items);
        ICharacterBuilder PerformDeed(string deed); // Загальний метод для справ
        Character GetCharacter();
    }
}
using System;
using Task5.Builders;
using Task5.Directors;
using Task5.Products;

namespace Task5
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            Director director = new Director();

            // 1. Створюємо нашого героя
            HeroBuilder heroBuilder = new HeroBuilder();
            director.ConstructDreamHero(heroBuilder);
            Character dreamHero = heroBuilder.GetCharacter();
            dreamHero.ShowInfo();

            // 2. Створюємо ворога
            EnemyBuilder enemyBuilder = new EnemyBuilder();
            director.ConstructArchenemy(enemyBuilder);
            Character archenemy = enemyBuilder.GetCharacter();
            archenemy.ShowInfo();

            // 3. Демонстрація гнучкості Будівельника (можемо працювати без Директора)
            Console.WriteLine("\n=== КАСТОМНИЙ NPC (Без Директора) ===");
            HeroBuilder customBuilder = new HeroBuilder();
            customBuilder.Reset()
                         .SetName("Звичайний мандрівник")
                         .SetAppearance("Низький", "Худий", "Світле", "Блакитні")
                         .SetClothing("Плащ")
                         .AddInventory(new System.Collections.Generic.List<string> { "Карта",
"Яблуко" });
```

```
            customBuilder.GetCharacter().ShowInfo();

            Console.ReadLine();
        }
    }
}
```

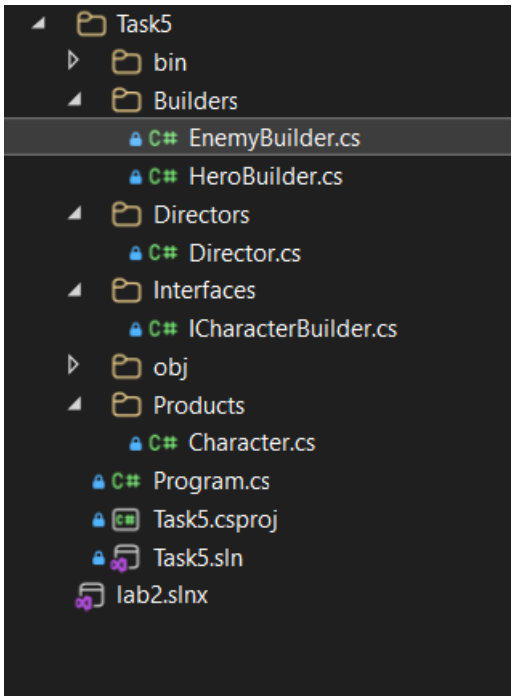


Рис. 10

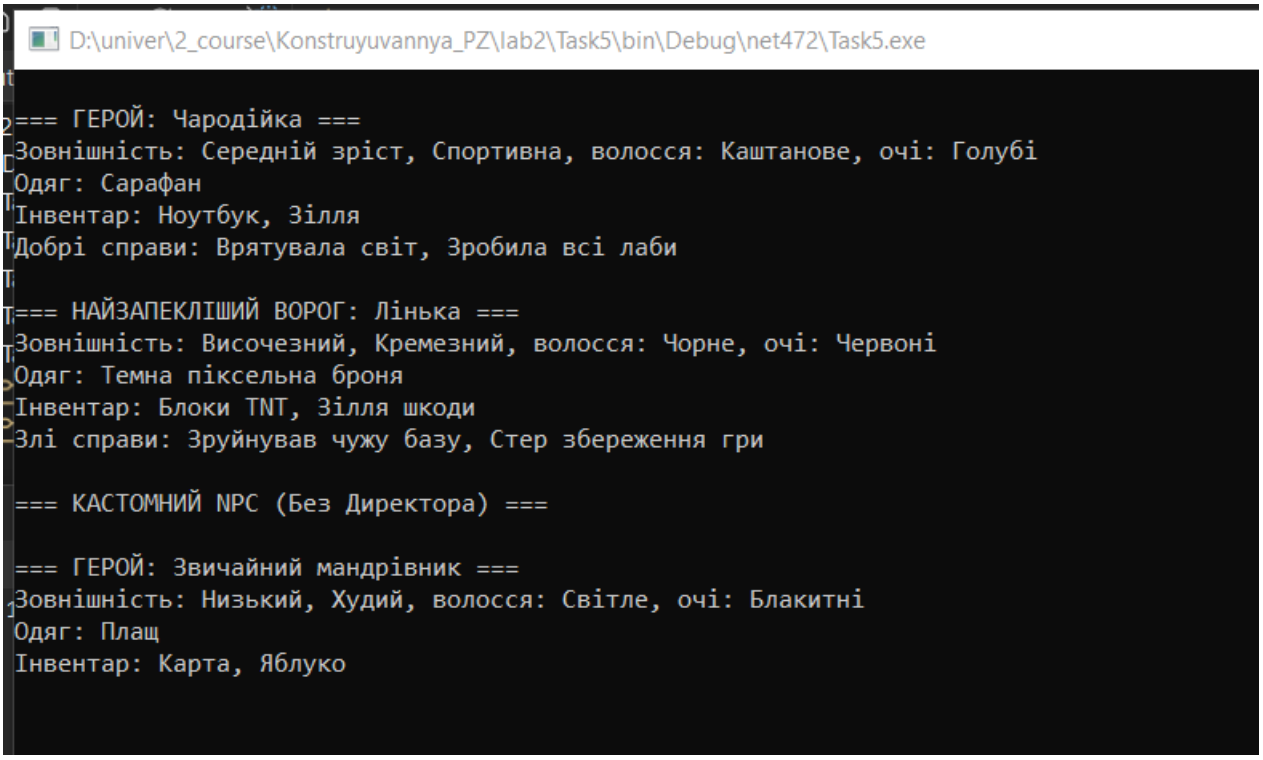


Рис. 12